Table of Contents

ITPRA0-22 Project 2
1. Learner Management 2
2. Inheritance 3
3. Encapsulation 5
4. Assessment and Marks 5
5. Exception Handling & GUI Requirement 6

Figure 1: Adding a learner 2
Figure 2: Adding , updating , removing learner 3
Figure 3: result of adding ,removing, updating learner details 3
Figure 4: Derived class using inheritance 4
Figure 5: internal representation is hidden using encapsulation 5
Figure 6: checking if student passed/failed or certified 5
Figure 7: inserting instructions on how to add student details 6
Figure 8: formulating conditions 6

ITPRA0-22 Project

Written by Maureen Ngwenya

- Student No: eduv4955953

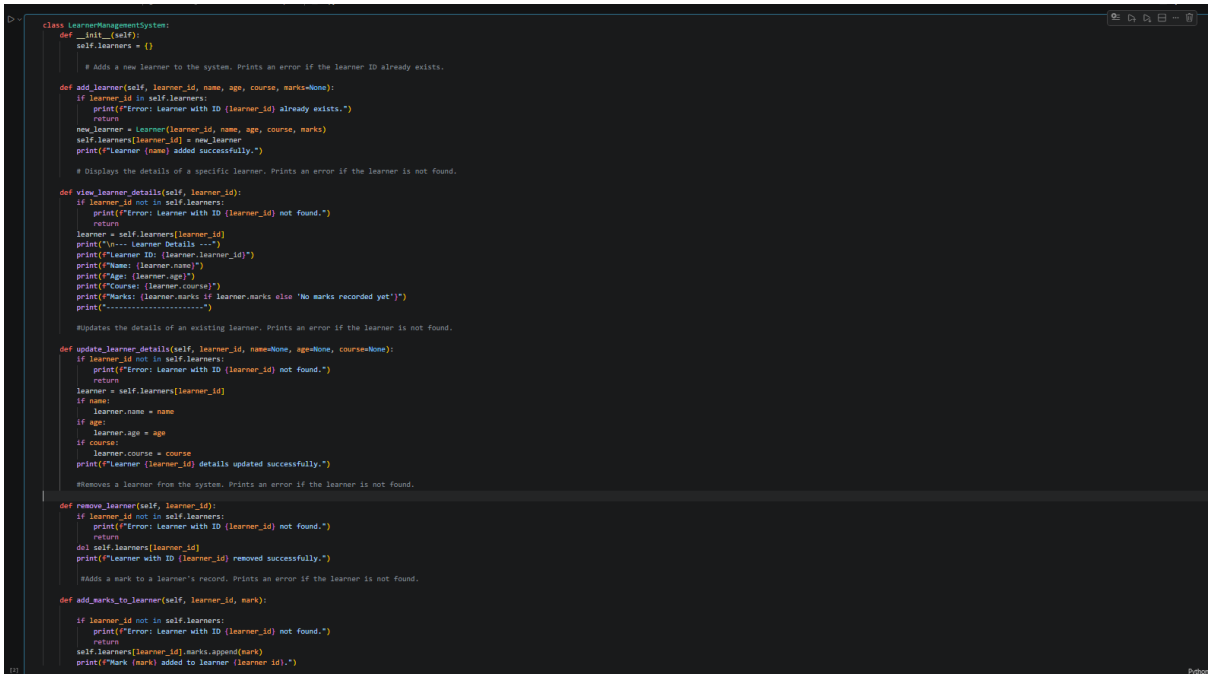
- Date: 2026-06-09

The purpose of this assignment is to develop a **Python-based Learner Progress Tracking System (LPTS)** that helps a community learning centre manage learner information, record assessment marks, calculate averages, determine pass or fail status, and generate progress feedback. The assignment also demonstrates the application of key Python programming concepts such as variables, input/output, decision-making, loops, functions, lists, object-oriented programming, inheritance, encapsulation, recursion, exception handling, and simple GUI message boxes.

1. Learner Management



Figure 1: Adding a learner



```
class LearnerManagementSystem:
    def __init__(self):
        self.learners = {}

        # Adds a new learner to the system. Prints an error if the learner ID already exists.

    def add_learner(self, learner_id, name, age, course, marks=None):
        if learner_id in self.learners:
            print(f"Error: Learner with ID {learner_id} already exists.")
            return
        new_learner = Learner(learner_id, name, age, course, marks)
        self.learners[learner_id] = new_learner
        print(f"Learner {name} added successfully.")

        # Displays the details of a specific learner. Prints an error if the learner is not found.

    def view_learner_details(self, learner_id):
        if learner_id not in self.learners:
            print(f"Error: Learner with ID {learner_id} not found.")
            return
        learner = self.learners[learner_id]
        print("\n--- Learner Details ---")
        print(f"Learner ID: {learner.learner_id}")
        print(f"Name: {learner.name}")
        print(f"Age: {learner.age}")
        print(f"Course: {learner.course}")
        print(f"Marks: {learner.marks if learner.marks else 'No marks recorded yet'}")
        print("-----------------------")

        #Updates the details of an existing learner. Prints an error if the learner is not found.

    def update_learner_details(self, learner_id, name=None, age=None, course=None):
        if learner_id not in self.learners:
            print(f"Error: Learner with ID {learner_id} not found.")
            return
        learner = self.learners[learner_id]
        if name:
            learner.name = name
        if age:
            learner.age = age
        if course:
            learner.course = course
        print(f"Learner {learner_id} details updated successfully.")

        #Removes a learner from the system. Prints an error if the learner is not found.

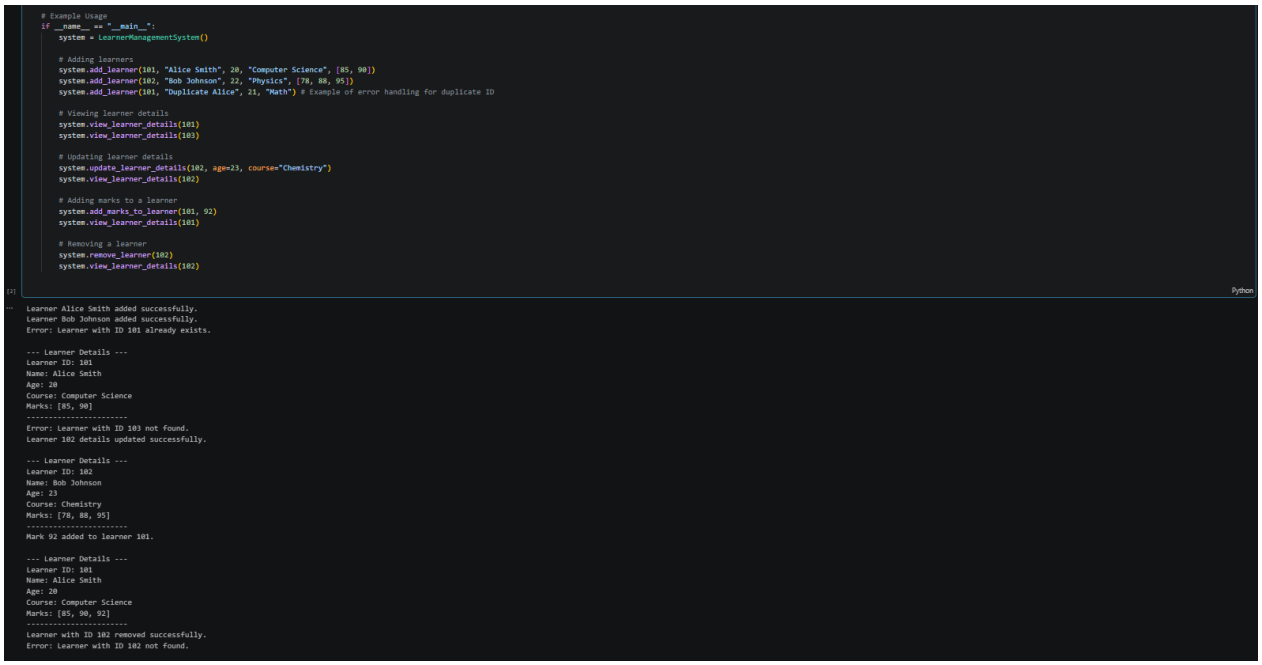
    def remove_learner(self, learner_id):
        if learner_id not in self.learners:
            print(f"Error: Learner with ID {learner_id} not found.")
            return
        del self.learners[learner_id]
        print(f"Learner with ID {learner_id} removed successfully.")

        #Adds a mark to a learner's record. Prints an error if the learner is not found.

    def add_marks_to_learner(self, learner_id, mark):

        if learner_id not in self.learners:
            print(f"Error: Learner with ID {learner_id} not found.")
            return
        self.learners[learner_id].marks.append(mark)
        print(f"Mark {mark} added to learner {learner_id}.")
```

Figure 2: Adding , updating , removing learner



```
# Example Usage
if __name__ == "__main__":
    system = LearnerManagementSystem()

    # Adding learners
    system.add_learner(101, "Alice Smith", 20, "Computer Science", [85, 90])
    system.add_learner(102, "Bob Johnson", 22, "Physics", [78, 88, 95])
    system.add_learner(101, "Duplicate Alice", 21, "Math") # Example of error handling for duplicate ID

    # Viewing learner details
    system.view_learner_details(101)
    system.view_learner_details(103)

    # Updating learner details
    system.update_learner_details(102, age=23, course="Chemistry")
    system.view_learner_details(102)

    # Adding marks to a learner
    system.add_marks_to_learner(101, 92)
    system.view_learner_details(101)

    # Removing a learner
    system.remove_learner(102)
    system.view_learner_details(102)
```

```
Learner Alice Smith added successfully.
Learner Bob Johnson added successfully.
Error: Learner with ID 101 already exists.

--- Learner Details ---
Learner ID: 101
Name: Alice Smith
Age: 20
Course: Computer Science
Marks: [85, 90]
-----------------------
Error: Learner with ID 103 not found.
Learner 102 details updated successfully.

--- Learner Details ---
Learner ID: 102
Name: Bob Johnson
Age: 23
Course: Chemistry
Marks: [78, 88, 95]
-----------------------
Mark 92 added to learner 101.

--- Learner Details ---
Learner ID: 101
Name: Alice Smith
Age: 20
Course: Computer Science
Marks: [85, 90, 92]
-----------------------
Learner with ID 102 removed successfully.
Error: Learner with ID 102 not found.
```

Figure 3: result of adding ,removing, updating learner details

2. Inheritance

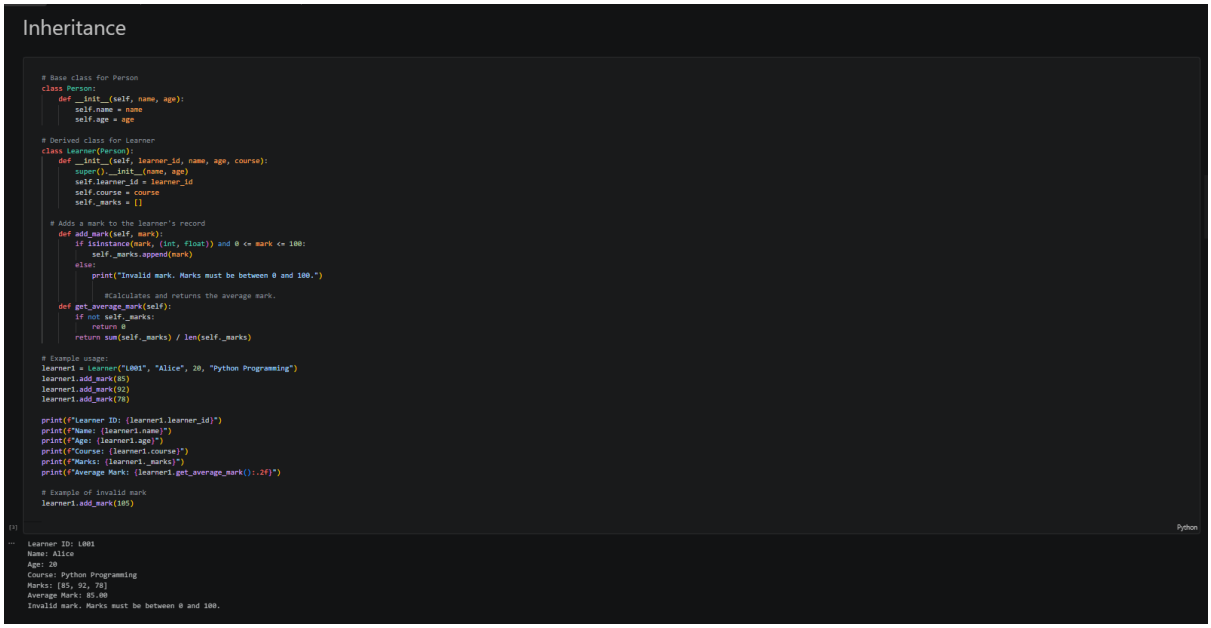


Figure 4: Derived class using inheritance

3. Encapsulation

Encapsulation

```
    def add_mark(self, mark):
        if 0 <= mark <= 100:
            self._marks.append(mark)
        else:
            raise ValueError("Invalid mark entry. Marks must be between 0 and 100.")

    def get_average(self):
        if not self._marks:
            return 0
        return self._sum_marks(self._marks) / len(self._marks)

    def _sum_marks(self, marks):
        if not marks:
            return 0
        return marks[0] + self._sum_marks(marks[1:])

    # Example usage
    learner1 = Learner("L001", "Sarah",17,'Python')

    learner1.add_mark(75)
    learner1.add_mark(85)
    learner1.add_mark(90)

    print("Learner 1 average:", learner1.get_average_mark())

    learner2= Learner("L002", "Jones",18,'Python')

    learner2.add_mark(95)
    learner2.add_mark(65)
    learner2.add_mark(50)

    print("Learner 2 average:", learner2.get_average_mark())
```

Python

```
Learner 1 average: 83.33333333333333
Learner 2 average: 70.0
```

Figure 5: internal representation is hidden using encapsulation

4. Assessment and Marks

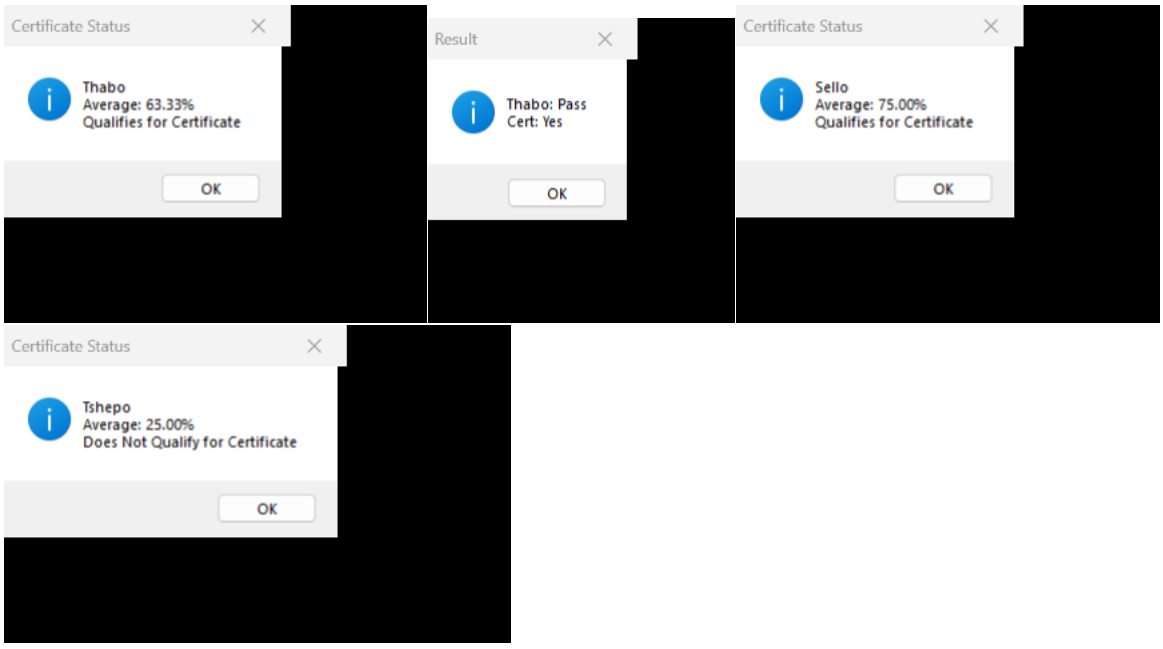
```
Success: Learner Thabo added successfully!
<__main__.Learner object at 0x0000016D86549550>

--- Learner Summary ---
ID: 1 | Name: Thabo
Course: Data analytics
Marks: [65, 80, 45]
Average: 63.33
Result: Pass
Qualifies for Certificate: Yes
Success: Learner Sello added successfully!
<__main__.Learner object at 0x0000016D86587390>

--- Learner Summary ---
ID: 2 | Name: Sello
Course: Data analytics
Marks: [90, 90, 45]
Average: 75.00
Result: Pass with Distinction
Qualifies for Certificate: Yes
Success: Learner Tshepo added successfully!
<__main__.Learner object at 0x0000016D86587250>

--- Learner Summary ---
ID: 3 | Name: Tshepo
Course: Data analytics
Marks: [20, 10, 45]
Average: 25.00
Result: Fail
Qualifies for Certificate: No
```

Figure 6: checking if student passed/failed or certified



5. Exception Handling & GUI Requirement

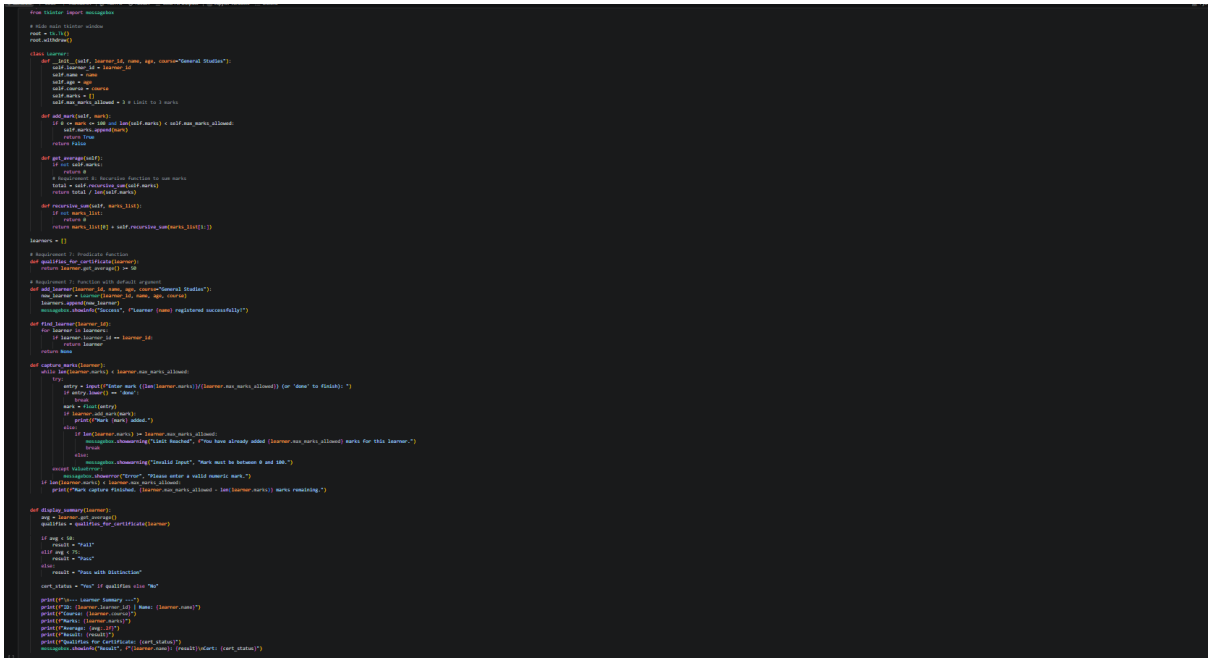


Figure 7: inserting instructions on how to add student details



Figure 8: formulating conditions



